

JOÃO ANDRADE

ROGER ROCHA

SAMAR VICTOR

SIMULADOR DE ESCALONAMENTO DE TAREFAS
DOCUMENTAÇÃO DO PROJETO

São Bernardo do Campo - SP

2026

JOÃO ANDRADE
ROGER ROCHA
SAMAR VICTOR

SIMULADOR DE ESCALONAMENTO DE TAREFAS
DOCUMENTAÇÃO DO PROJETO

Documentação do projeto desenvolvida para a disciplina de Sistemas Operacionais, no 8º semestre, sob responsabilidade do professor Vinicius Borges.

São Bernardo do Campo - SP

2026

SUMÁRIO

1 VISÃO GERAL	--
2 SEPARAÇÃO ENTRE POLÍTICA E MECANISMO	--
3 DIAGRAMA DE MÓDULOS	--
4 ESTRUTURA DE UMA TAREFA	--
5 INTERFACE DOS MÓDULOS	--
6 PARÂMETROS CONFIGURÁVEIS	--
7 FUNCIONAMENTO INTERNO	--
8 CONVENÇÕES DE SIMULAÇÃO	--
9 DEPENDÊNCIAS E AMBIENTE	--

1 VISÃO GERAL

O projeto implementa um simulador de escalonamento de tarefas em tempo discreto. Cada tarefa possui instante de ingresso, tempo de processamento, prioridade e, opcionalmente, uma seção crítica associada ao recurso exclusivo R. O simulador executa uma unidade de trabalho por vez, registra a linha do tempo e calcula métricas ao final da execução.

São implementadas as políticas FCFS, SJF, SRTF, Round-Robin, prioridade cooperativa e prioridade preemptiva. O mecanismo também representa custo de troca de contexto, bloqueio por recurso, inversão de prioridades, herança de prioridade, teto de prioridade e envelhecimento.

2 SEPARAÇÃO ENTRE POLÍTICA E MECANISMO

A arquitetura separa a decisão de qual tarefa deve executar do mecanismo que mantém o estado da simulação. `control/motor.py` administra relógio, tarefas prontas, troca de contexto, recurso R, bloqueios, envelhecimento e métricas. `control/politicas.py` contém apenas os critérios específicos de escolha das seis políticas.

A cada unidade de tempo, o motor monta o conjunto de tarefas prontas. Esse conjunto é entregue à política, que devolve a tarefa escolhida. O motor não depende do critério de ordenação da política, enquanto a política não controla diretamente o relógio, o recurso ou a contabilização das métricas.

3 DIAGRAMA DE MÓDULOS

Figura 1 - Fluxo de dados entre as camadas do simulador

Fluxo principal de dados do simulador

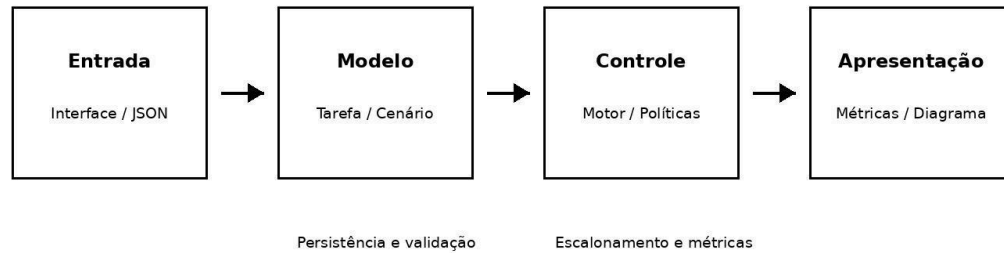


Tabela 1 - Módulos e responsabilidades

Camada	Arquivos principais	Responsabilidade
Entrada	view/janela.py e arquivos JSON	Recebe tarefas e parâmetros e aciona as operações.
Modelo	model/tarefa.py; model/cenario.py	Representa, valida, grava e carrega os dados.
Controle	control/motor.py; control/politicas.py; control/gerador.py	Executa a simulação, escolhe políticas, sorteia e compara cenários.
Apresentação	view/relatorio.py e view/janela.py	Formata métricas e apresenta o diagrama de tempo.

4 ESTRUTURA DE UMA TAREFA

Tabela 2 - Campos da classe Tarefa

Campo	Tipo	Função
identificador	int	Identificador inteiro positivo.
ingresso	int	Instante não negativo em que a tarefa surge.
tp	int	Tempo de processamento positivo.
prioridade	int	Prioridade base; valor maior representa prioridade mais alta.
sc_inicio	int ou None	Posição da solicitação de R no tempo de execução próprio.
sc_duracao	int	Duração da seção crítica; vale zero quando R não é utilizado.

A validação impede identificadores menores que 1, ingresso negativo, tempo de processamento não positivo, prioridade menor que 1 e seções críticas que ultrapassem a duração da tarefa. Quando sc_inicio não é informado, sc_duracao deve ser zero.

5 INTERFACE DOS MÓDULOS

Tabela 3 - Interfaces principais do código

Módulo	Interface principal	Função
model/tarefa.py	Tarefa; validar(); clonar(); para_dicionario(); de_dicionario()	Representa e valida uma tarefa e permite sua serialização.
model/cenario.py	gravar(); carregar()	Persiste e recupera tarefas e parâmetros em JSON.
control/politicas.py	Politica.chave(); criar()	Define os critérios das políticas e cria uma política pela sigla.
control/motor.py	Escalonador.executar(); prioridade_efetiva()	Mantém o laço de simulação, o recurso, os protocolos e as métricas.
control/gerador.py	sortear_cenario(); comparar(); rodar_lote()	Gera cenários e executa comparações e lotes.
view/janela.py	criar_janela()	Constrói a interface e encaminha as ações do usuário.
view/relatorio.py	formatação e diagrama	Apresenta métricas, comparativos e a linha do tempo.

6 PARÂMETROS CONFIGURÁVEIS

Tabela 4 - Parâmetros do escalonador

Parâmetro	Faixa / opções	Padrão ou regra
quantum tq	Inteiro; usado apenas no RR	No RR, deve ser maior que ttc.
custo_troca ttc	Inteiro maior ou igual a 0	0
alfa	Inteiro maior ou igual a 0	0
protocolo	nenhum, heranca ou teto	nenhum
% com recurso	0 a 100	Usado somente no sorteio de cenários.
amostras	Inteiro maior ou igual a 1	Usado somente na execução em lote.

7 FUNCIONAMENTO INTERNO

7.1 Laço principal

O método `Escalonador.executar()` mantém o instante atual e repete o ciclo até que todas as tarefas sejam concluídas. Em cada iteração, os novos ingressos são registrados, o conjunto de prontas é montado e a política escolhe a próxima tarefa. Quando nenhuma tarefa está pronta, o relógio avança até o próximo evento de ingresso.

Quando a tarefa despachada é diferente da última ocupante do processador, uma troca de contexto é contabilizada. Se `ttc` for maior que zero, o intervalo correspondente é registrado na linha do tempo. No Round-Robin, o custo da troca também é descontado da fatia disponível.

7.2 Recurso exclusivo e suspensão

A tarefa solicita R quando sua quantidade já executada alcança `sc_inicio`. Se R estiver livre, a tarefa torna-se sua detentora. Se estiver ocupado, a solicitante é marcada como suspensa e deixa o conjunto de tarefas prontas até que o recurso seja liberado.

7.3 Herança, teto e envelhecimento

Com herança, o detentor de R recebe temporariamente a maior prioridade base observada entre as tarefas suspensas. Com teto, a prioridade do detentor é elevada, na aquisição, ao maior valor de prioridade entre as tarefas que declaram uso de R. Nos dois casos, a prioridade base é restaurada após a liberação do recurso.

O envelhecimento é calculado pela prioridade atual somada a α multiplicado pelo tempo decorrido desde a referência de espera. A referência corresponde ao último despacho ou, quando ainda não houve despacho, ao instante de ingresso.

7.4 Métricas

Tabela 5 - Métricas calculadas

Métrica	Definição
Tt	conclusão - ingresso.
Tp	tempo de processamento original.
Tw	$Tt - Tp$.
Primeira execução	primeira_execucao - ingresso.
Médias	Média aritmética das métricas entre as tarefas do cenário.
Eficiência	$tq/(tq + ttc)$, quando existe quantum.

8 CONVENÇÕES DE SIMULAÇÃO

As convenções abaixo determinam a reprodução dos resultados. Alterações nessas decisões podem produzir números diferentes para o mesmo conjunto de tarefas.

Tabela 6 - Convenções adotadas

Código	Convenção
C1	O tempo é discreto e uma unidade útil é executada por iteração.
C2	Valores maiores representam prioridades mais altas.
C3	O desempate utiliza menor ingresso e, em seguida, menor identificador.
C4	A troca de contexto é contabilizada quando a tarefa despachada difere da última ocupante do processador.
C5	O custo da troca é descontado da fatia concedida no Round-Robin.
C6	Ao esgotar o quantum, a tarefa retorna à cauda depois das tarefas que ingressaram no mesmo instante.
C7	A seção crítica é medida no tempo de execução próprio da tarefa.
C8	T_w é calculado como $T_t - T_p$.
C9	O teto do recurso é calculado entre as tarefas que declaram seção crítica, mesmo que não ocorra disputa.
C10	O envelhecimento toma como referência o último despacho ou, na ausência deste, o ingresso.

9 DEPENDÊNCIAS E AMBIENTE

A aplicação requer Python 3.10 ou superior. O código utiliza módulos da biblioteca padrão e Tkinter para a interface gráfica. Não há dependências externas instaladas por gerenciador de pacotes.

A entrega não contém executável independente. No Windows, EXECUTAR.bat posiciona o diretório de trabalho na pasta do projeto e inicia main.pyw com pythonw; caso essa abertura falhe, o script tenta main.py. No macOS ou Linux, EXECUTAR.command posiciona o diretório e executa python3 main.py.

A suíte de testes é executada pelo comando `python -m unittest discover -s tests -t .` e contém 38 testes automatizados. Na revisão desta documentação, os 38 testes foram executados com resultado OK.

Para estender o simulador com uma nova política, cria-se uma classe derivada de Política em `control/politicas.py`, implementa-se o método `chave()` e registra-se a classe no dicionário ALGORITMOS. O mecanismo comum permanece em `control/motor.py`.